https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/

CPP:-

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr)
{}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) :
val(x), left(left), right(right) {}
 * };
 */

    class Solution {
public:
    vector<vector<int> > zigzagLevelOrder(TreeNode* root) {
    if (root == NULL) {
        return vector<vector<int> > ();
    }
    vector<vector<int> > result;

    queue<TreeNode*> nodesQueue;
    nodesQueue.push(root);
    bool leftToRight = true; //flag

    while ( !nodesQueue.empty()) {
        int size = nodesQueue.size();//number of nodes of a
level
        vector<int> row(size);
        for (int i = 0; i < size; i++) {
            TreeNode* node = nodesQueue.front();
```

```
            nodesQueue.pop();

            // find position to fill node's value
            int index = (leftToRight) ? i : (size - 1 - i);

            row[index] = node->val;
            if (node->left) {
                nodesQueue.push(node->left);
            }
            if (node->right) {
                nodesQueue.push(node->right);
            }
        }
        // after this level
        leftToRight = !leftToRight;
        result.push_back(row);
    }
    return result;
}
};
```

Java :-

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
```

```
 * }
 */
class Solution {
    public ArrayList<ArrayList<Integer>>
zigzagLevelOrder(TreeNode root) {

        Queue <TreeNode>q=new LinkedList<TreeNode>();

         ArrayList<ArrayList<Integer>> ans= new ArrayList<>();

        boolean flag =true;

        if(root==null) return ans;

        q.offer(root);

        while(!q.isEmpty()){

            int size=q.size();
            ArrayList<Integer> s= new ArrayList<>();

            for(int i=0;i<size;i++)
            {
                int index=i;
                if(q.peek().left!=null)
q.offer(q.peek().left);
                if(q.peek().right!=null)
q.offer(q.peek().right);
                if(flag==true) s.add(q.poll().val);
                else
                    s.add(0,q.poll().val);

            }
            flag=!flag;
            ans.add(s);
        }
        return ans;
```

```
    }
}
```

https://www.geeksforgeeks.org/problems/bottom-view-of-binary-tree/1

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <queue>
#include <map>

using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    // Constructor to initialize
    // the node with a value
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution{
public:
    // Function to return the
    // bottom view of the binary tree
    vector<int> bottomView(Node* root){
        // Vector to store the result
        vector<int> ans;

        // Check if the tree is empty
        if(root == NULL){
            return ans;
        }

        // Map to store the bottom view nodes
        // based on their vertical positions
        map<int, int> mpp;

        // Queue for BFS traversal, each
        // element is a pair containing node
        // and its vertical position
```

```
        queue<pair<Node*, int>> q;

        // Push the root node with its vertical
        // position (0) into the queue
        q.push({root, 0});

        // BFS traversal
        while(!q.empty()){
            // Retrieve the node and its vertical
            // position from the front of the queue
            auto it = q.front();
            q.pop();
            Node* node = it.first;
            int line = it.second;

            // Update the map with the node's data
            // for the current vertical position
            mpp[line] = node->data;

            // Process left child
            if(node->left != NULL){
                // Push the left child with a decreased
                // vertical position into the queue
                q.push({node->left, line - 1});
            }

            // Process right child
            if(node->right != NULL){
                // Push the right child with an increased
                // vertical position into the queue
                q.push({node->right, line + 1});
            }
        }

        // Transfer values from the
        // map to the result vector
        for(auto it : mpp){
            ans.push_back(it.second);
        }

        return ans;
    }
};

int main() {
    // Creating a sample binary tree
```

```
    Node* root = new Node(1);
    root->left = new Node(2);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);
    root->right = new Node(3);
    root->right->right = new Node(10);
    root->right->left = new Node(9);

    Solution solution;

    // Get the Bottom View traversal
    vector<int> bottomView =
                    solution.bottomView(root);

    // Print the result
    cout << "Bottom View Traversal: "<< endl;
    for(auto node: bottomView){
        cout << node << " ";
    }

    return 0;
}
```

Java :-

```
// Node structure for the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    public Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}

public class Solution {
    // Function to return the
    // bottom view of the binary tree
```

```java
public List<Integer> bottomView(Node root) {
    // List to store the result
    List<Integer> ans = new ArrayList<>();

    // Check if the tree is empty
    if(root == null) {
        return ans;
    }

    // Map to store the bottom view nodes
    // based on their vertical positions
    Map<Integer, Integer> mpp = new TreeMap<>();

    // Queue for BFS traversal, each
    // element is a pair containing node
    // and its vertical position
    Queue<Pair<Node, Integer>> q = new LinkedList<>();

    // Push the root node with its vertical
    // position (0) into the queue
    q.add(new Pair<>(root, 0));

    // BFS traversal
    while(!q.isEmpty()) {
        // Retrieve the node and its vertical
        // position from the front of the queue
        Pair<Node, Integer> pair = q.poll();
        Node node = pair.getKey();
        int line = pair.getValue();

        // Update the map with the node's data
        // for the current vertical position
        mpp.put(line, node.data);

        // Process left child
        if(node.left != null) {
            // Push the left child with a decreased
            // vertical position into the queue
            q.add(new Pair<>(node.left, line - 1));
        }

        // Process right child
        if(node.right != null) {
            // Push the right child with an increased
            // vertical position into the queue
            q.add(new Pair<>(node.right, line + 1));
```

```
            }
        }

        // Transfer values from the
        // map to the result list
        for(Map.Entry<Integer, Integer> entry : mpp.entrySet()) {
            ans.add(entry.getValue());
        }

        return ans;
    }

    public static void main(String[] args) {
        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(4);
        root.left.right = new Node(10);
        root.left.left.right = new Node(5);
        root.left.left.right.right = new Node(6);
        root.right = new Node(3);
        root.right.right = new Node(10);
        root.right.left = new Node(9);

        Solution solution = new Solution();

        // Get the Bottom View traversal
        List<Integer> bottomView = solution.bottomView(root);

        // Print the result
        System.out.println("Bottom View Traversal: ");
        for(int node : bottomView) {
            System.out.print(node + " ");
        }
    }
}
```

https://www.geeksforgeeks.org/problems/top-view-of-binary-tree/1

```
#include <iostream>
#include <vector>
#include <set>
#include <queue>
#include <map>
```

```cpp
using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    // Constructor to initialize
    // the node with a value
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution{
public:
    // Function to return the
    // top view of the binary tree
    vector<int> topView(Node* root){
        // Vector to store the result
        vector<int> ans;

        // Check if the tree is empty
        if(root == NULL){
            return ans;
        }

        // Map to store the top view nodes
        // based on their vertical positions
        map<int, int> mpp;

        // Queue for BFS traversal, each element
        // is a pair containing node
        // and its vertical position
        queue<pair<Node*, int>> q;

        // Push the root node with its vertical
        // position (0) into the queue
        q.push({root, 0});

        // BFS traversal
        while(!q.empty()){
            // Retrieve the node and its vertical
            // position from the front of the queue
            auto it = q.front();
            q.pop();
            Node* node = it.first;
```

```
            int line = it.second;

            // If the vertical position is not already
            // in the map, add the node's data to the map
            if(mpp.find(line) == mpp.end()){
                mpp[line] = node->data;
            }

            // Process left child
            if(node->left != NULL){
                // Push the left child with a decreased
                // vertical position into the queue
                q.push({node->left, line - 1});
            }

            // Process right child
            if(node->right != NULL){
                // Push the right child with an increased
                // vertical position into the queue
                q.push({node->right, line + 1});
            }
        }

        // Transfer values from the
        // map to the result vector
        for(auto it : mpp){
            ans.push_back(it.second);
        }

        return ans;
    }
};

int main() {
    // Creating a sample binary tree
    Node* root = new Node(1);
    root->left = new Node(2);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);
    root->right = new Node(3);
    root->right->right = new Node(10);
    root->right->left = new Node(9);

    Solution solution;
```

```
    // Get the top view traversal
    vector<int> topView =
                    solution.topView(root);

    // Print the result
    cout << "Top View Traversal: "<< endl;
    for(auto node: topView){
        cout << node << " ";
    }

    return 0;
}
```

Java :-

```
import java.util.*;

// Node structure for the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    public Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}

public class Solution {
    // Function to return the
    // top view of the binary tree
    public List<Integer> topView(Node root) {
        // List to store the result
        List<Integer> ans = new ArrayList<>();

        // Check if the tree is empty
        if (root == null) {
            return ans;
        }
```

```
        // Map to store the top view nodes
        // based on their vertical positions
        Map<Integer, Integer> mpp = new TreeMap<>();

        // Queue for BFS traversal, each element
        // is a pair containing node
        // and its vertical position
        Queue<Pair<Node, Integer>> q = new LinkedList<>();

        // Push the root node with its vertical
        // position (0) into the queue
        q.add(new Pair<>(root, 0));

        // BFS traversal
        while (!q.isEmpty()) {
            // Retrieve the node and its vertical
            // position from the front of the queue
            Pair<Node, Integer> pair = q.poll();
            Node node = pair.getKey();
            int line = pair.getValue();

            // If the vertical position is not already
            // in the map, add the node's data to the map
            if (!mpp.containsKey(line)) {
                mpp.put(line, node.data);
            }

            // Process left child
            if (node.left != null) {
                // Push the left child with a decreased
                // vertical position into the queue
                q.add(new Pair<>(node.left, line - 1));
            }

            // Process right child
            if (node.right != null) {
                // Push the right child with an increased
                // vertical position into the queue
                q.add(new Pair<>(node.right, line + 1));
            }
        }

        // Transfer values from the
        // map to the result list
        for (int value : mpp.values()) {
            ans.add(value);
```

```
        }

        return ans;
    }

    public static void main(String[] args) {
        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(4);
        root.left.right = new Node(10);
        root.left.left.right = new Node(5);
        root.left.left.right.right = new Node(6);
        root.right = new Node(3);
        root.right.right = new Node(10);
        root.right.left = new Node(9);

        Solution solution = new Solution();

        // Get the top view traversal
        List<Integer> topView = solution.topView(root);

        // Print the result
        System.out.println("Vertical Traversal: ");
        for (int node : topView) {
            System.out.print(node + " ");
        }
    }
}
```